ALGORITMUS DE CASTELJAU
SUBDIVISIONS
BÉZIEROVY PLÁTY

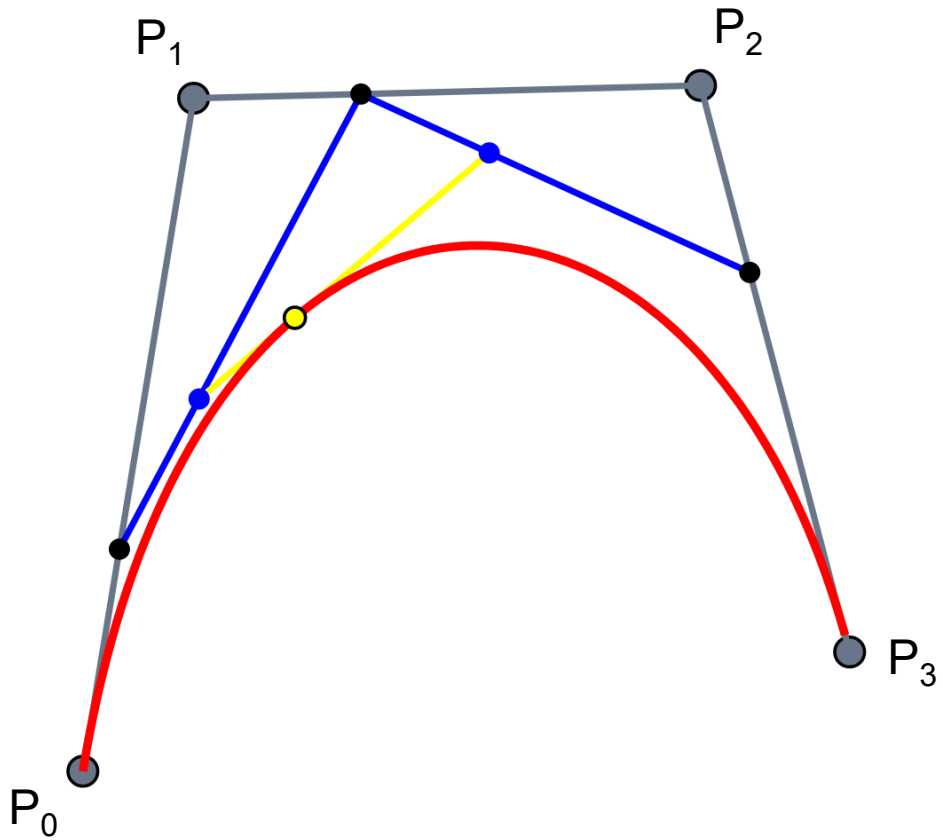
Kurz: **Moderní počítačová grafika**

Lektor: Ing. Michal Švento

Náplň cvičení

1. Implementace algoritmu de Casteljau pro Bézierovu kubiku – **bodovaný úkol**
2. Subdivisions algoritmem de Casteljau
3. Bézierovy pláty
4. Subdivisions plátů

Algoritmus de Casteljau



https://www.utko.fekt.vut.cz/~rajmic/applets/deCasteljau/deCasteljau.html

Algoritmus de Casteljau

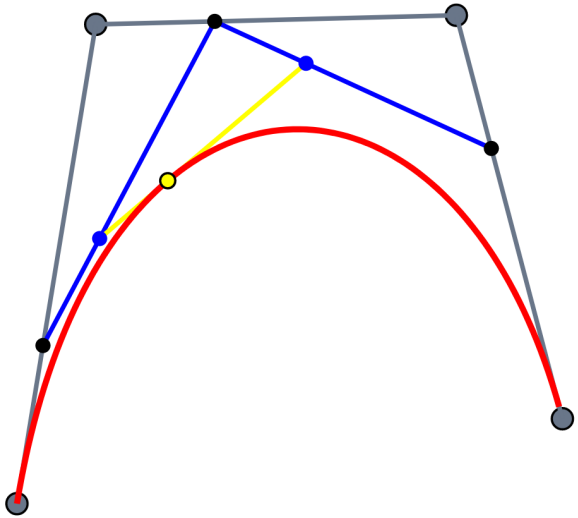
- mějme dáno N řídících bodů
- algoritmus má $(N-1)$ iterací
- v i-té iteraci pro čas t počítáme

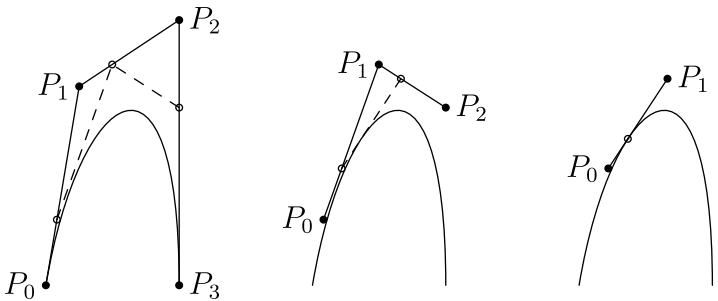
iterace iterace iterace

$$P_j^i = (1-t)P_j^{i-1} + tP_{j+1}^{i-1}$$

index bodu index bodu index bodu

- v každé iteraci dostaneme o jeden řídící bod méně (poslední „zahazujeme“)
- v poslední iteraci získáváme jediný bod – bod ležící na křivce

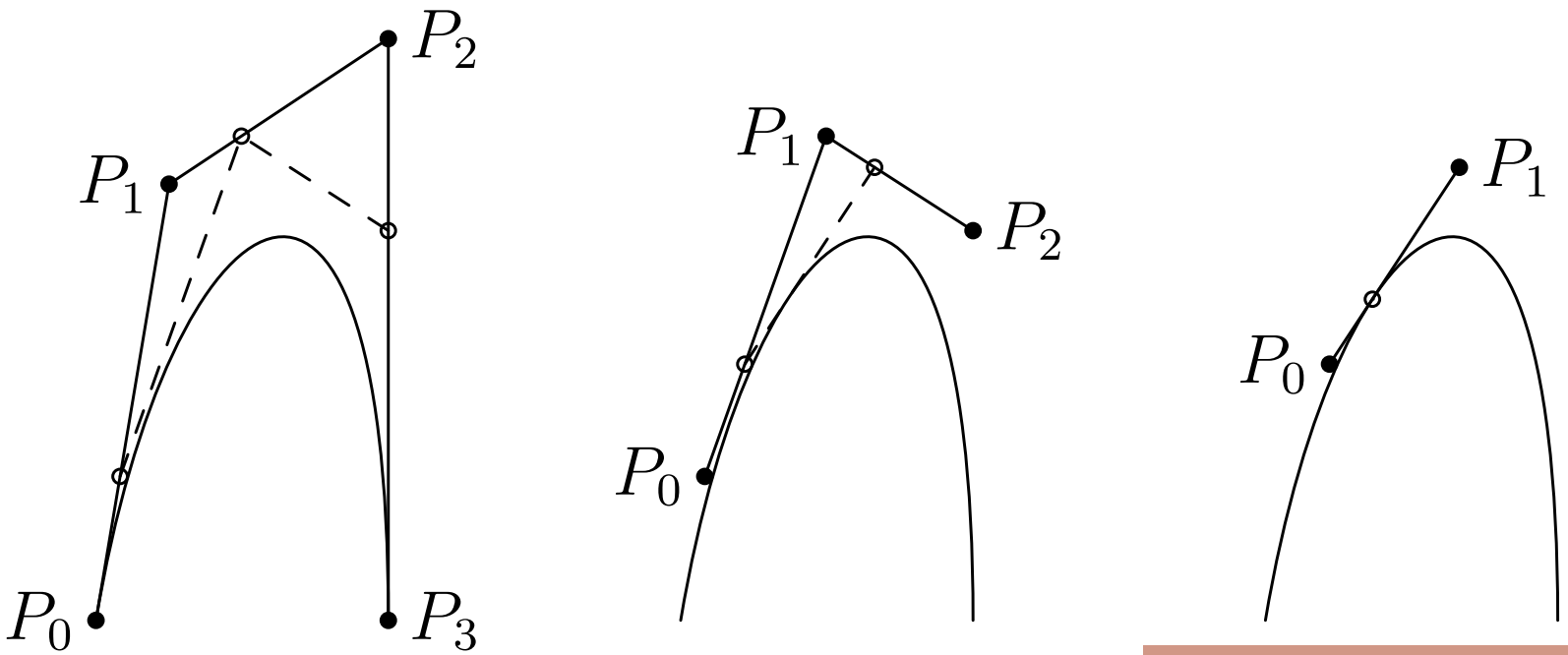




Úkol 1

bodovaný úkol

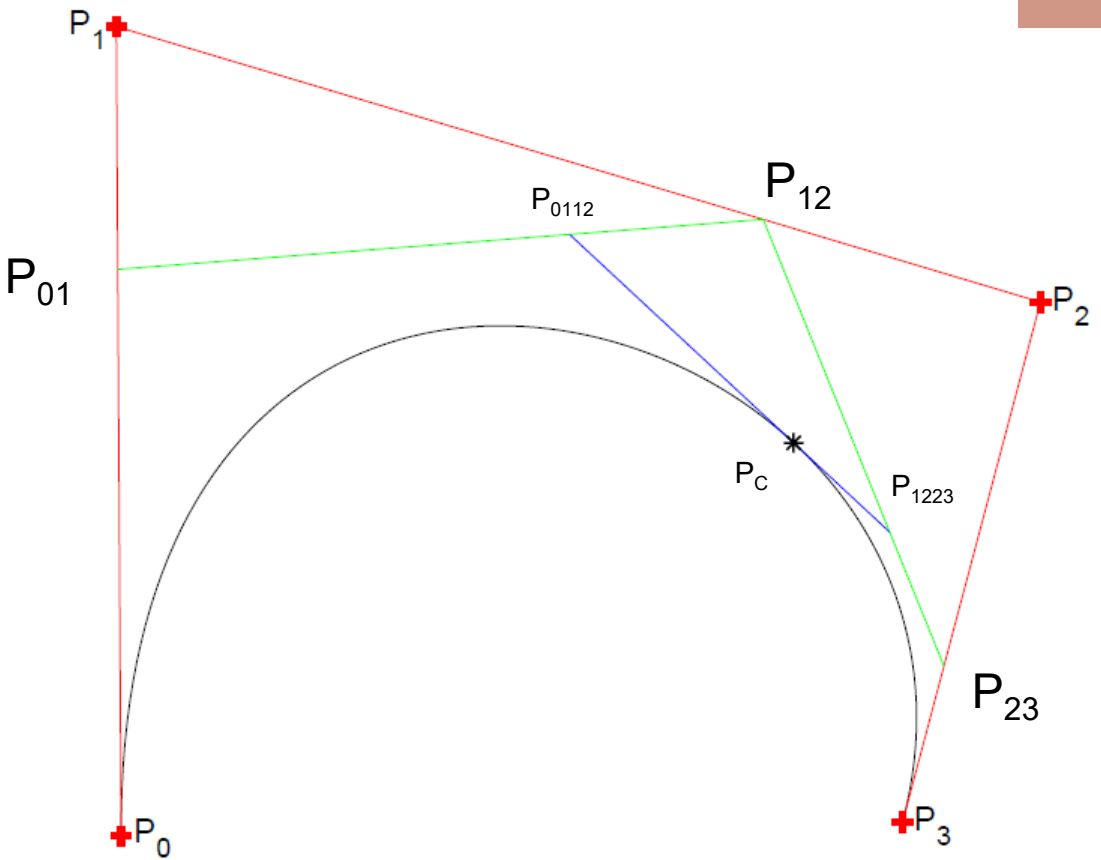
1. Doplňte chybějící kód ve funkci `deCasteljau()` pro generování bodů Bézierovy kubiky algoritmem de Casteljau.



`01_deCasteljau.cpp`

Úkol 1

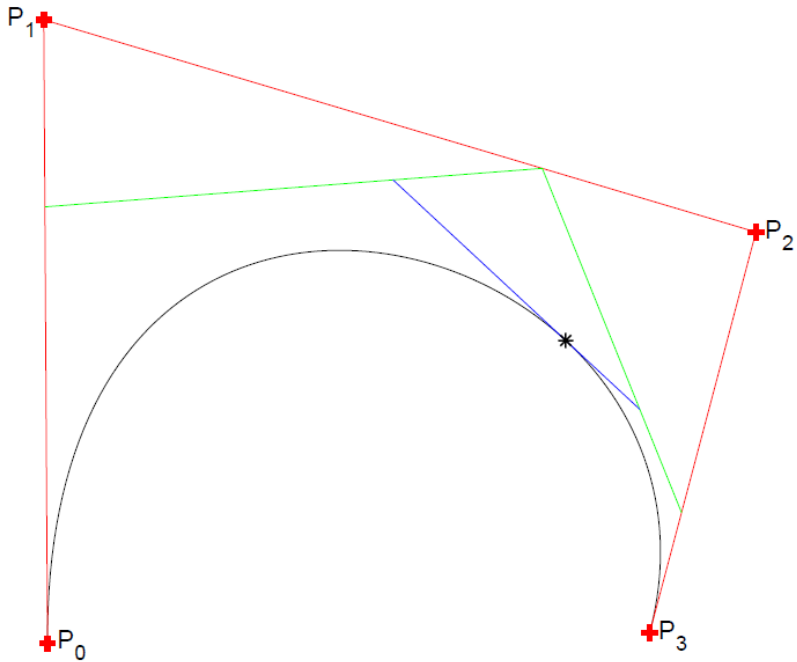
bodovaný úkol

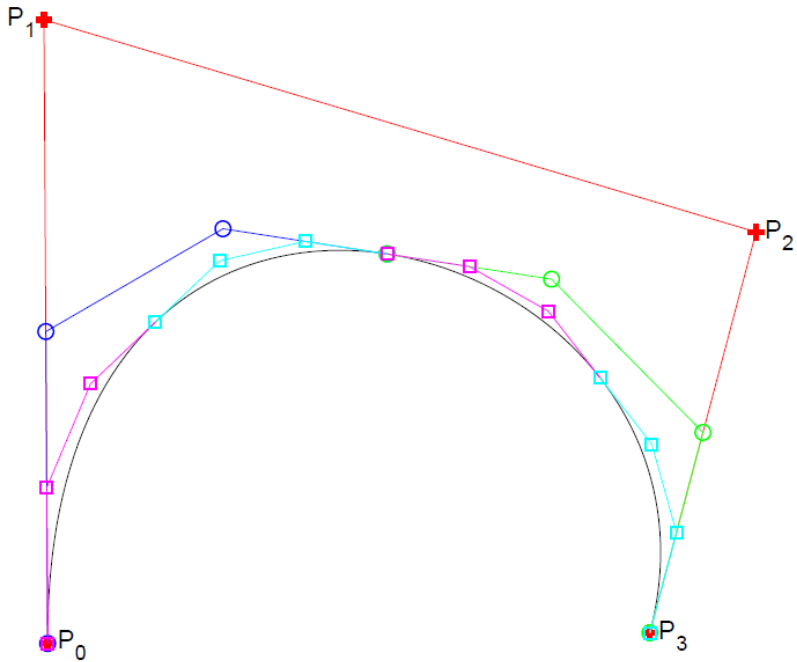


01_deCasteljau.cpp

Subdivisions

- pomocí algoritmů jako jsou Catmull–Clark, Doo–Sabin, Loop,... lze iterativním způsobem dosáhnout hladšího povrchu objektů



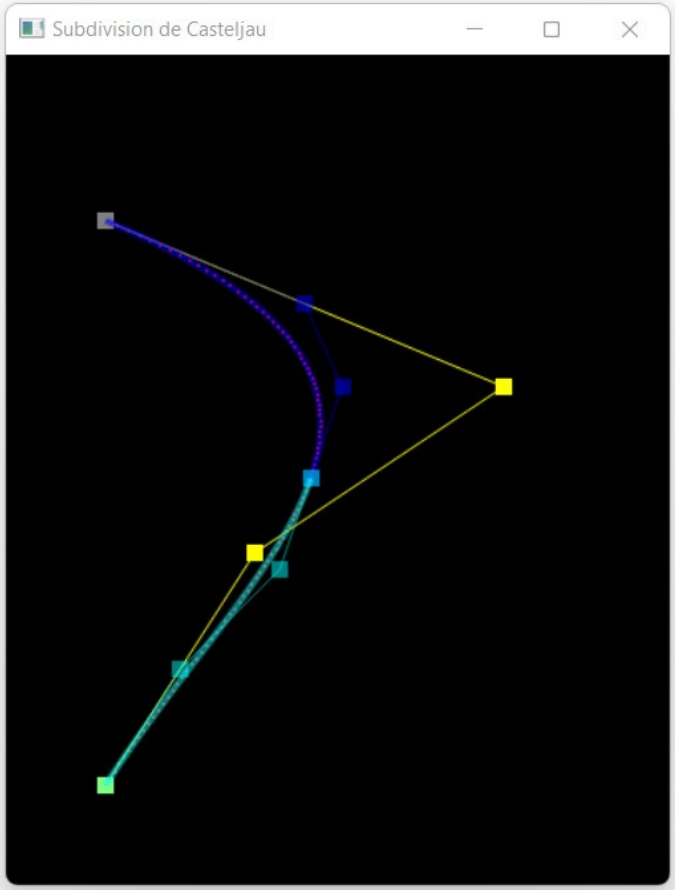


Úkol 2

1. Realizovat subdivision zadané křivky metodou de Casteljau
2. Vykreslit:
   - původní křivku + její řídicí body
   - obě dvě nové křivky + jejich řídicí body
3. Scéna půjde měnit pomocí myši, reagovat budou ale pouze řídicí body původní křivky
   - Subdivision se aktualizuje automaticky

`02_subdivision.cpp`

Úkol 2



02_subdivision.cpp

Bézierovy pláty

- parametrické vyjádření

$$Q(u,v) = \begin{cases} x = x(u,v) \\ y = y(u,v) \\ z = z(u,v) \end{cases}$$

- tentokrát nemáme jednu proměnnou t znázorňující čas, ale dvě proměnné u, v

Bézierovy pláty

- výpočet nejčastěji používané bikubiky:

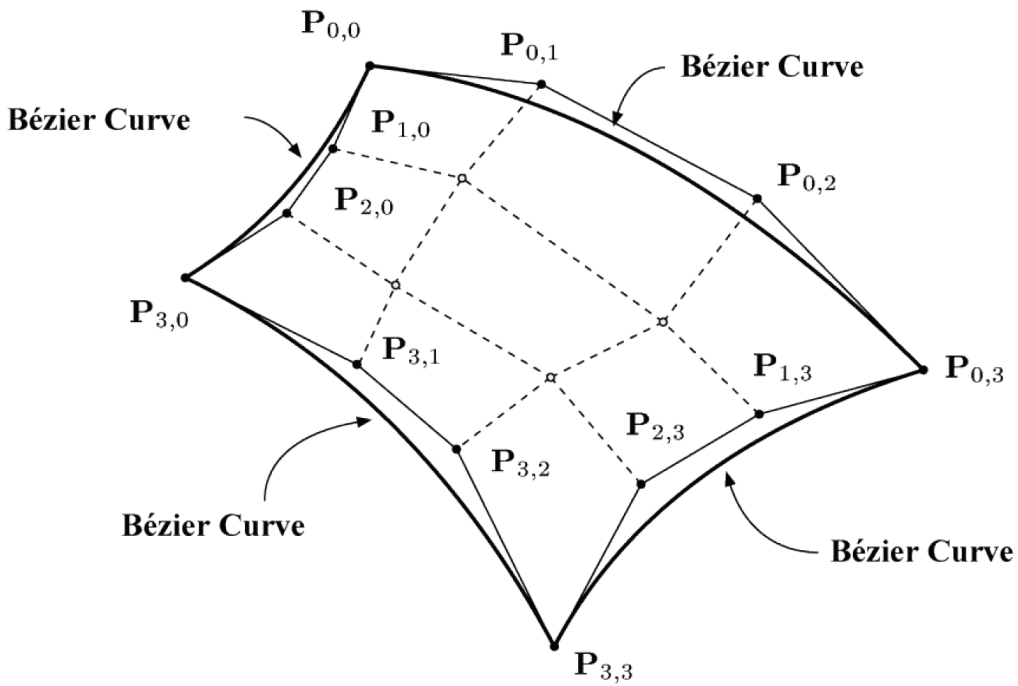
$$x(u,v) = \begin{bmatrix} u^3 & u^2 & u & 1 \end{bmatrix} \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \mathbf{G}_x \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} v^3 \\ v^2 \\ v \\ 1 \end{bmatrix}$$

$$y(u,v) = \begin{bmatrix} u^3 & u^2 & u & 1 \end{bmatrix} \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \mathbf{G}_y \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} v^3 \\ v^2 \\ v \\ 1 \end{bmatrix}$$

$$z(u,v) = \begin{bmatrix} u^3 & u^2 & u & 1 \end{bmatrix} \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \mathbf{G}_z \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} v^3 \\ v^2 \\ v \\ 1 \end{bmatrix}$$

Bézierovy pláty

- matice $\mathbf{G}_x, \mathbf{G}_y, \mathbf{G}_z$ jsou formátu 4×4 a obsahují příslušné souřadnice 16 řídících bodů $\mathbf{P}_{0,0}, \dots, \mathbf{P}_{3,3}$



Úkol 3

1. Ve funkci `onDisplay()` pomocí evaluátoru vykreslete plochu Bézierova plátu primitivou `GL_POINTS`.
2. Ve směru u i ve směru v vykreslete body s hustotou 50.

`03_BezieruvPlat.cpp`

Nápověda k evaluátoru OpenGL

- evaluátor pro 3D je již zapnutý pomocí

```
glEnable(GL_MAP2_VERTEX_3);
```

- nastavení evaluátoru

```
glMap2f(    GL_MAP2_VERTEX_3,
            GLfloat u1,
            GLfloat u2,
            GLint ustride,
            GLint uorder,
            GLfloat v1,
            GLfloat v2,
            GLint vstride,
            GLint vorder,
            const GLfloat * points    );
```

- výpočet bodu se provádí pomocí

```
glEvalCoord2f(u, v);
```

Nápověda k evaluátoru OpenGL

$u \rightarrow$

$v \downarrow$

```
float ridiciBody[4][4][3] = {
    { {-15, -15,  20},  {-5, -15,  40},  {5, -15,  40},  {15, -15,  20} },
    { {-15,  -5,  10},  {-5,  -5,  30},  {5,  -5,  30},  {15,  -5,  10} },
    { {-15,   5,   0},  {-5,   5,  20},  {5,   5,  20},  {15,   5,   0} },
    { {-15,  15, -20},  {-5,  15,   0},  {5,   15,  0},  {15,  15, -20} }  };
```

- `ustride` specifies the number of floats or doubles between the beginning of control point $\mathbf{P}_{i,j}$ and the beginning of control point $\mathbf{P}_{i+1,j}$
- `uorder` specifies the dimension of the control point array in the u axis

- `vstride` specifies the number of floats or doubles between the beginning of control point $\mathbf{P}_{i,j}$ and the beginning of control point $\mathbf{P}_{i,j+1}$
- `vorder` specifies the dimension of the control point array in the v axis

Úkol 4

Porovnejte, jaký efekt mají subdivisions na zobrazenou mřížku bodů s evaluátorem

03_BezieruvPlat.cpp